5.3 填充算法实现

```
/**
 * Span结构: 表示一个扫描线段
 */
struct Span {
    int x1, x2;  // 线段的左右边界
    int y;       // 扫描线的y坐标
    int dy;      // 扫描方向(1:向下, -1:向上)
};

/**
 * 扫描线种子填充算法
 * @param x, y 种子点坐标
 * @param targetColor 目标颜色(要被填充的颜色)
 * @param fillColor 填充颜色
 */
void FloodFillScanline(int x, int y, UINT32 targetColor, UINT32
   fillColor) {
    std::queue<Span> spanQueue;  // 待处理的扫描线段队列

    // 从种子点向左右扩展, 找到当前扫描线上的可填充区间
    int xl = x, xr = x;
    while (xl > 0 && GetPixelColor(xl - 1, y) == targetColor) xl--;
    while (xr < width - 1 && GetPixelColor(xr + 1, y) == targetColor)
        xr++;

    // 填充当前扫描线段
    for (int i = xl; i <= xr; i++) SetPixelColor(i, y, fillColor);

    // 将上下两个方向的扫描线段加入队列
    spanQueue.push({xl, xr, y, 1});   // 向下扫描
    spanQueue.push({xl, xr, y, -1});  // 向上扫描

    // 处理队列中的所有扫描线段
    while (!spanQueue.empty()) {
        Span span = spanQueue.front();
        spanQueue.pop();

        int ny = span.y + span.dy;  // 相邻扫描线的y坐标
```